

Sistema de Gestión de Biblioteca Universitaria

Documento de Especificación del Proyecto

Asignatura: Lenguaje de Programación 2

Docente: Ing. Bryan Vélez M.Sc.

Grupo: #1

Semestre: Tercero — Modalidad en Línea

Universidad: Universidad Agraria del Ecuador (UAE)

Fecha: Julio 2026

Índice

1. 1. Introducción
2. 2. Justificación
3. 3. Objetivos
4. 4. Alcance
5. 5. Marco Teórico
6. 6. Metodología de Desarrollo
7. 7. Diseño Preliminar
8. 8. Distribución del Trabajo
9. 9. Repositorio y Control de Versiones
10. 10. Plan de Trabajo y Cronograma

1. Introducción

Las bibliotecas universitarias enfrentan el desafío de gestionar eficientemente sus recursos bibliográficos: registro de socios, control de préstamos, devoluciones y reservas. Actualmente, muchas bibliotecas pequeñas y medianas aún utilizan métodos manuales o sistemas genéricos que no se adaptan a sus necesidades específicas.

El presente proyecto tiene como objetivo desarrollar un Sistema de Gestión de Biblioteca Universitaria que automatice los procesos diarios de una biblioteca física. El sistema permitirá registrar socios (estudiantes y docentes), administrar el catálogo de libros, controlar préstamos y devoluciones, y gestionar reservas de ejemplares no disponibles.

El sistema se desarrolla como proyecto final de la asignatura Lenguaje de Programación 2, aplicando los conceptos de Programación Orientada a Objetos, estructuras de datos, algoritmos de ordenamiento y búsqueda, bases de datos SQLite e interfaz gráfica con Tkinter, todos enseñados durante el semestre por el Ing. Bryan Vélez.

2. Justificación

La Universidad Agraria del Ecuador, en su modalidad en línea, requiere que los estudiantes demuestren competencias en el desarrollo de software aplicando los principios de la programación estructurada y orientada a objetos. Este proyecto integra los conocimientos adquiridos durante el semestre en un producto funcional.

La automatización de una biblioteca universitaria permite:

- Reducir errores humanos en el registro manual de préstamos y devoluciones
- Agilizar la búsqueda de libros en el catálogo
- Mantener un control preciso de los ejemplares disponibles
- Mejorar la experiencia del usuario (socios y bibliotecarios)

3. Objetivos

3.1 Objetivo General

Desarrollar un sistema de escritorio para la gestión de una biblioteca universitaria que permita administrar socios, libros, préstamos y reservas, aplicando los principios de la Programación Orientada a Objetos, estructuras de datos, algoritmos de búsqueda y ordenamiento, y persistencia en base de datos SQLite.

3.2 Objetivos Específicos

11. Implementar un modelo de clases utilizando herencia, encapsulamiento, polimorfismo y abstracción para representar los actores del sistema (Persona → Estudiante/Docente, Libro, Préstamo, Reserva).

12. Utilizar estructuras de datos lineales (lista enlazada, cola FIFO, pila LIFO) para la gestión dinámica de la información en memoria.
13. Implementar algoritmos de ordenamiento (burbuja, inserción, merge sort, quick sort) y búsqueda (lineal, binaria) para la manipulación del catálogo.
14. Persistir los datos en una base de datos SQLite con operaciones CRUD completas y claves foráneas.
15. Desarrollar una interfaz gráfica de usuario (GUI) utilizando Tkinter para la interacción con el sistema.

4. Alcance

4.1 Funcionalidades incluidas

- Gestión de Socios: registro, consulta y actualización de datos de estudiantes y docentes
- Gestión de Libros: alta, consulta y eliminación de libros en el catálogo
- Préstamos: realización de préstamos y registro de devoluciones
- Reservas: creación de reservas para libros no disponibles (cola de espera FIFO)
- Búsqueda: búsqueda de libros por título, autor, y aplicación de búsqueda binaria
- Ordenamiento: ordenamiento del catálogo por diferentes criterios usando múltiples algoritmos

4.2 Funcionalidades excluidas

- No incluye módulo de multas o sanciones
- No incluye generación de reportes estadísticos avanzados
- No incluye integración con sistemas externos (API web, RFID, etc.)

5. Marco Teórico

5.1 Programación Orientada a Objetos (POO)

La POO es un paradigma de programación que organiza el código en clases y objetos. Los cuatro pilares fundamentales son:

- Encapsulamiento: ocultación de datos mediante atributos privados (self.__atributo) y acceso controlado por getters/setters.
- Herencia: una clase puede heredar atributos y métodos de otra clase padre. Estudiante y Docente heredan de Persona.
- Polimorfismo: el mismo método puede comportarse de manera diferente según la clase. tipo_socio() retorna "Estudiante" o "Docente".
- Abstracción: se definen interfaces o métodos abstractos que las subclases deben implementar.

Referencia: Joyanes Aguilar, L. (2020). Programación orientada a objetos con Python. McGraw-Hill Interamericana.

5.2 Estructuras de Datos

5.2.1 Lista Enlazada Simple

Estructura lineal donde cada elemento (nodo) contiene un valor y un puntero al siguiente nodo. Se implementó LinkedList como base para la cola y la pila.

Referencia: Weiss, M. A. (2013). Estructuras de datos y algoritmos (4ª ed.). Pearson Educación.

5.2.2 Cola (Queue — FIFO)

El primer elemento en entrar es el primero en salir. Se utiliza para gestionar las reservas.

5.2.3 Pila (Stack — LIFO)

El último en entrar es el primero en salir. Se utiliza para deshacer operaciones.

5.3 Algoritmos de Ordenamiento y Búsqueda

5.3.1 Ordenamiento

Algoritmo	Complejidad	Descripción
Bubble Sort	$O(n^2)$	Compara pares adyacentes y los intercambia
Insertion Sort	$O(n^2)$	Inserta cada elemento en su posición correcta
Merge Sort	$O(n \log n)$	Divide, ordena recursivamente y fusiona
Quick Sort	$O(n \log n)$	Selecciona pivote y particiona

5.3.2 Búsqueda

Algoritmo	Complejidad	Requisito
Búsqueda Lineal	$O(n)$	Ninguno
Búsqueda Binaria	$O(\log n)$	Lista ordenada

5.4 Base de Datos SQLite

SQLite es un motor de base de datos embebido que no requiere servidor. El sistema utiliza `sqlite3.connect()`, `cursor.execute()` con parámetros posicionales (`?, ?`), y `PRAGMA foreign_keys = ON` para claves foráneas.

5.5 Interfaz Gráfica con Tkinter

Tkinter es la biblioteca gráfica estándar de Python. Se utiliza para construir la ventana principal, menús desplegables, formularios y cuadros de diálogo. Demostrado en clase por el Ing. Bryan Vélez (10-jul, timestamp 54:30).

6. Metodología de Desarrollo

Se utiliza el Modelo Incremental, que consiste en desarrollar el sistema por etapas o incrementos, donde cada incremento agrega funcionalidad completa al producto. Este modelo permite:

- Entregar versiones funcionales tempranas
- Obtener retroalimentación continua
- Reducir riesgos de integración
- Adaptarse a cambios durante el desarrollo

6.1 Incrementos del Proyecto

Incremento	Semana	Entregable	Estado
Incremento 1	Semana 12 (7-10 jul)	Modelo POO + Estructuras + Algoritmos + BD + GUI (stubs) + Documento 50%	☐ Completado
Incremento 2	Semana 13 (13-17 jul)	GUI funcional + BD poblada + CRUD + Pruebas + Manual de Usuario	☐ Pendiente
Incremento 3	Semana 14 (20 jul)	Integración final + Documento 100% + Defensa	☐ Pendiente

6.2 Flujo de trabajo

16. Planificación del incremento: se definen las funcionalidades a implementar
17. Diseño: se actualizan los diagramas si es necesario
18. Implementación: cada miembro desarrolla su módulo asignado
19. Pruebas: se ejecutan pruebas unitarias y de integración
20. Revisión: el líder (Henry) revisa e integra los cambios
21. Entrega: se sube a GitHub y se documenta el avance

6.3 Herramientas de desarrollo

Herramienta	Versión	Propósito
Python	3.11+	Lenguaje de programación
VS Code	Última	Editor con Python + GitLens
Tkinter	8.6	Interfaz gráfica
SQLite3	3.x	Base de datos embebida
DB Browser	3.x	Admin. visual BD
Git	2.47+	Control de versiones
GitHub	—	Repositorio remoto

7. Diseño Preliminar

7.1 Diagrama de Clases (UML)

El sistema utiliza una jerarquía de clases donde Persona es la clase base abstracta. Estudiante y Docente heredan de Persona. Socio es un wrapper polimórfico. Libro, Prestamo y Reserva completan el modelo del dominio.

7.2 Modelo Entidad-Relación

La base de datos consta de 4 tablas relacionadas: socios, libros, prestamos (FK a socio y libro) y reservas (FK a socio y libro).

8. Distribución del Trabajo

8.1 Roles del equipo

Integrante	Rol	Módulo	Archivos
Henry Pazmiño	Líder / Integrador	Modelos POO + Integración	main.py, src/models/*
Jodie Parrales	Desarrolladora GUI	Interfaz Tkinter	src/gui/app.py
Cindy Ayoví	Desarrolladora BD	Base de datos SQLite	src/database/*
Cesar Gonzales	Desarrollador Lógica	Algoritmos + Estructuras	src/algorithms/*, src/data_structures/*
Mayra Vera	Documentación y Pruebas	Pruebas + Documentación	tests/*, docs/*

9. Repositorio y Control de Versiones

9.1 Repositorio

URL: <https://github.com/josuepaz80-usitee/sistema-biblioteca-prog2>

Rama principal: main

9.2 Historial de Commits

#	Fecha	Hash	Mensaje
1	7 jul	0e78b3a	Initial commit
2	7 jul	b31c6bd	feat: estructura inicial del Sistema de Gestion de Biblioteca Universitaria
3	8 jul	eaf6cae	refactor: estilo nivel 3er semestre segun lo ensenado

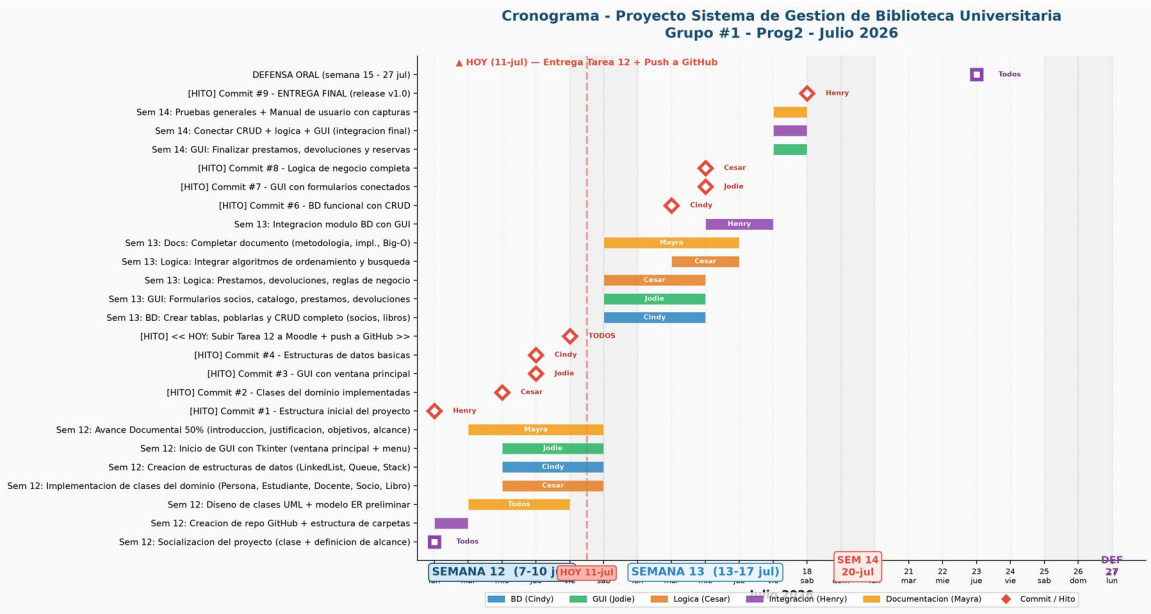
			por Ing. Bryan Velez + DECLARATORIA
4	9 jul	7f9e535	fix: DECLARATORIA actualizada tras revisión completa del material
5	9 jul	d388507	fix: referencias genericas sin nombres individuales en DECLARATORIA
6	11 jul	ac193dd	docs: agregar documentación del proyecto (especificación, manual, Gantt, README)
7	11 jul	6cf6167	docs: especificación visible en markdown + créditos de autoría por integrante en cada archivo
8	11 jul	2d1e028	fix: formato limpio del crédito de Mayra en manual- usuario.md
9	11 jul	e8ce1b1	fix: actualizar historial de commits y DECLARATORIA (Tkinter + incremental)

9.3 Stack Tecnológico

- Editor: VS Code con extensión Python (Microsoft) + GitLens
- Control de versiones: Git + GitHub
- Lenguaje: Python 3.11+
- GUI: Tkinter (librería estándar de Python)
- BD: SQLite3
- Documentación: Markdown + DOCX + PDF

10. Plan de Trabajo y Cronograma

10.1 Diagrama de Gantt



10.2 Prioridades para la Semana 13 (13-17 julio)

Prioridad	Tarea	Responsable
☐ Alta	Implementar formularios CRUD en GUI (Registrar Socio, Agregar Libro)	Jodie Parrales
☐ Alta	Poblar la base de datos con datos de prueba	Cindy Ayoví
☐ Media	Completar lógica de préstamos y devoluciones en GUI	Jodie + Cesar
☐ Media	Pruebas de integración módulo BD + GUI	Mayra Vera
☐ Baja	Manual de usuario con capturas de pantalla	Mayra Vera

Documento de Especificación — Avance 50%

Grupo #1 — Lenguaje de Programación 2 — UAE